Assignment 2 – Zero-Shot Image Classification with Transformers

In this assignment, you will apply a pre-trained vision–language transformer (e.g. CLIP) to perform **zero-shot** classification on the Fashion-MNIST dataset—classifying each image without any task-specific training. You will build on the concepts from Assignment 1 by comparing this "off-the-shelf" approach to the CNN you previously trained.

You will:

1. **Load** the Fashion-MNIST images using PyTorch instead of Keras.
2. **Run a zero-shot baseline** with simple text prompts to set a performance reference.
3. **Engineer improved prompts** and measure the resulting accuracy gains.
4. **Visualise image embeddings** with UMAP to inspect class separability.
5. **Conduct one mini-experiment** of your choice.
6. **Summarise findings** and reflect on strengths and weaknesses of zero-shot transformers versus a trained CNN.

1. Loading the Fashion-MNIST Dataset

As in assignment 1, we'll load the Fashion-MNIST dataset, but this time using `torchvision.datasets` to ensure compatibility with the `transformers` library. We will also load our model and processor from the `transformers` library.

The transformers library allows us to use pre-trained models like CLIP, which can perform zero-shot classification by leveraging the text prompts we provide. There are two key objects we will use: the `CLIPModel` for the model itself and the `CLIPProcessor` for preparing our images and text prompts.

Since we are not actually training a model in this assignment, we will set the CLIP model to evaluation mode. If the model is designed to utilize features like dropout or batch normalization, setting it to evaluation mode ensures that these features behave correctly during inference (prediction). Setting the model to evaluaton mode also tells PyTorch that we don't have to compute gradients, which can save memory and speed up inference.

In order to speed up processing, we will also move the model to an "accelerator" if available. This is typically a GPU, but modern MacBooks also have an "Apple Silicon" accelerator that can be used for inference, called MPS (Metal Performance Shaders). If you are using a MacBook with Apple Silicon, you can use the MPS device for faster processing.

```
# Uncomment and run if required
#!pip install transformers torchvision torch accelerate


from transformers import CLIPModel, CLIPProcessor
import torch

clip_model_name = "openai/clip-vit-base-patch32"
clip_model     = CLIPModel.from_pretrained(clip_model_name)
clip_processor = CLIPProcessor.from_pretrained(clip_model_name, use_fast=False)

# Set model to evaluation mode, as we are not training it
clip_model.eval()

# Check for accelerators
device = "cpu" # Default to CPU
if torch.cuda.is_available():
    device = "cuda" # Use GPU if available
elif torch.backends.mps.is_available():
    device = "mps"

clip_model.to(device)

print(f"Using device: {device}")
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public model
  warnings.warn(
config.json:        4.19k/? [00:00<00:00, 129kB/s]
pytorch_model.bin: 100%                                605M/605M [00:15<00:00, 83.7MB/s]
model.safetensors: 100%                                605M/605M [00:11<00:00, 91.2MB/s]
preprocessor_config.json: 100%                         316/316 [00:00<00:00, 5.46kB/s]
tokenizer_config.json: 100%                            592/592 [00:00<00:00, 15.7kB/s]
vocab.json:         862k/? [00:00<00:00, 2.79MB/s]
merges.txt:         525k/? [00:00<00:00, 1.73MB/s]
special_tokens_map.json: 100%                          389/389 [00:00<00:00, 33.4kB/s]
tokenizer.json:        2.22M/? [00:00<00:00, 2.76MB/s]
Using device: cpu
```

Now we are ready to load the testing set from Fashion-MNIST. We will use the `torchvision.datasets.FashionMNIST` class to load the dataset. We do not need to apply any transformations to the images, as the `CLIPProcessor` ensures any input images are in the format that the model is trained on.

You should:

- ☐ Use the `torchvision.datasets.FashionMNIST` class to load the *test* split of the dataset. Documentation is available here.
- ☐ Create a PyTorch `DataLoader` to iterate over the dataset in batches. Use a batch size of 16 and set `shuffle=True` to randomise the order of the images. You will also need to supply the provided `collate_clip` function to the `DataLoader` `collate_fn` argument to ensure the images are processed correctly. Documentation for `DataLoader` is available here.

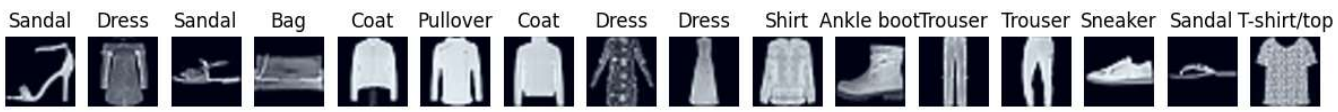
```
from torchvision import datasets
from torch.utils.data import DataLoader

CLASS_NAMES = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

def collate_clip(batch):
    imgs, labels = zip(*batch) # Unzip the batch into images and labels
    proc = clip_processor(images=list(imgs),
                          return_tensors="pt",
                          padding=True) # Process images with CLIPProcessor
    # Send pixel_values to GPU/CPU now; labels stay on CPU for metrics
    return proc["pixel_values"].to(device), torch.tensor(labels)

test_dataset = datasets.FashionMNIST(
    root="./data",
    train=False,
    download=True,
    transform=None
)
test_loader = DataLoader(
    test_dataset,
    batch_size=16,
```

```
    shuffle=True,
    collate_fn=collate_clip
)
```

```
100%|██████████| 26.4M/26.4M [00:01<00:00, 19.1MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 303kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.56MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 10.8MB/s]
```

If your code is correct, the following cell should show the first batch of images from the Fashion-MNIST dataset:

```
import matplotlib.pyplot as plt

# Display the first batch of images from `test_loader`

def show_batch(loader):
    images, labels = next(iter(loader))
    images = images.cpu()  # Move images to CPU for plotting
    # Renormalize to [0, 1] for visualization
    images = (images - images.min()) / (images.max() - images.min())
    _, axes = plt.subplots(1, len(images), figsize=(15, 5))
    for ax, img, label in zip(axes, images, labels):
        ax.imshow(img.permute(1, 2, 0))
        ax.set_title(CLASS_NAMES[label.item()])
        ax.axis('off')
    plt.show()

show_batch(test_loader)
```



We're now ready to run our zero-shot classification baseline!

Brief Introduction to Zero-Shot Classification

In Assignment 1, we followed the typical machine-learning pipeline: we trained a CNN on the Fashion-MNIST dataset, using labelled examples to update the model's weights. While effective, that approach requires a curated, task-specific training set—a luxury you don't always have in practice.

Zero-shot classification flips the script. A large vision–language model (VLM) such as **CLIP** is first pre-trained on hundreds of millions of image–text pairs scraped from the web. Because it learns *joint* visual–textual embeddings, the model can later solve new tasks simply by "measuring" how similar an image is to a **text prompt** that describes each candidate class—without seeing a single task-labelled example.

How it works

1. Feed an image through CLIP's vision encoder → **image feature**.
2. Feed a textual prompt (e.g. "a photo of a sandal") through CLIP's text encoder → **text feature**.
3. Compute cosine similarity between the image feature and every class's text feature.
4. Pick the class whose prompt is most similar.

For our first attempt, we'll use the bare class names as prompts, e.g.:

- "T-shirt/top"
- "Trouser"

You should:

- ☐ Build embeddings: use the `get_text_embeddings` helper function to create text embeddings for the class names.
- ☐ Run inference: use the `get_image_embeddings` helper function to create image embeddings.
- ☐ Compute cosine similarity: complete and use the `get_cosine_similarity` helper function to compute the cosine similarity between the image and text embeddings.
- ☐ Make predictions: use the `get_predictions` helper function to get the predicted class for each image in the batch.

Note that for normalized vectors like the ones we are using, cosine similarity is equivalent to the dot product. This means we can use the handy formula `cosine_similarity = vector_a @ vector_b.T` to compute the similarity between the image and text embeddings.

```
def get_text_embeddings(class_names: list[str]) -> torch.Tensor:
    """    Get text embeddings for the given class names using CLIP.
    Args:
        class_names (list[str]): List of class names to encode.
    Returns:
        torch.Tensor: Normalized text embeddings for the class names.
    """
    tokenized = clip_processor(text=class_names,
                               padding=True,
                               return_tensors="pt").to(device)

    with torch.no_grad():
        text_embeddings = clip_model.get_text_features(**tokenized)
```

```
    text_feats = text_embeddings / text_embeddings.norm(dim=-1, keepdim=True)

    return text_feats

def get_image_embeddings(images: torch.Tensor) -> torch.Tensor:
    """    Get image embeddings for the given images using CLIP.
    Args:
        images (torch.Tensor): Batch of images to encode.
    Returns:
        torch.Tensor: Normalized image embeddings for the images.
    """
    with torch.no_grad():
        image_embeddings = clip_model.get_image_features(pixel_values=images)

    image_feats = image_embeddings / image_embeddings.norm(dim=-1, keepdim=True)

    return image_feats


import numpy as np

def get_cosine_similarity(image_feats: torch.Tensor, text_feats: torch.Tensor) -> np.ndarray
    """
    Compute cosine similarity between image features and text features.
    Args:
        image_feats (torch.Tensor): Image features of shape (N, D).
        text_feats (torch.Tensor): Text features of shape (M, D).
    Returns:
        numpy.ndarray: Cosine similarity matrix of shape (N, M), where N is the number of im
    """
    image_feats = image_feats.cpu()  # Ensure image features are on CPU
    text_feats = text_feats.cpu()    # Ensure text features are on CPU

    # Compute cosine similarity, which is the dot product of normalized vectors
    return np.dot(image_feats.numpy(), text_feats.numpy().T)

def get_predictions(similarity: np.ndarray) -> np.ndarray:
    """
    Get predictions based on cosine similarity scores.
    Args:
        similarity (numpy.ndarray): Cosine similarity matrix of shape (N, M), where N is the
    Returns:
        numpy.ndarray: Predicted class indices for each image, shape (N,).
    """
    # Get the index of the maximum similarity for each image
    return np.argmax(similarity, axis=1)
```

With these functions complete, you are ready to run the zero-shot classification baseline. Complete the code to follow these steps:

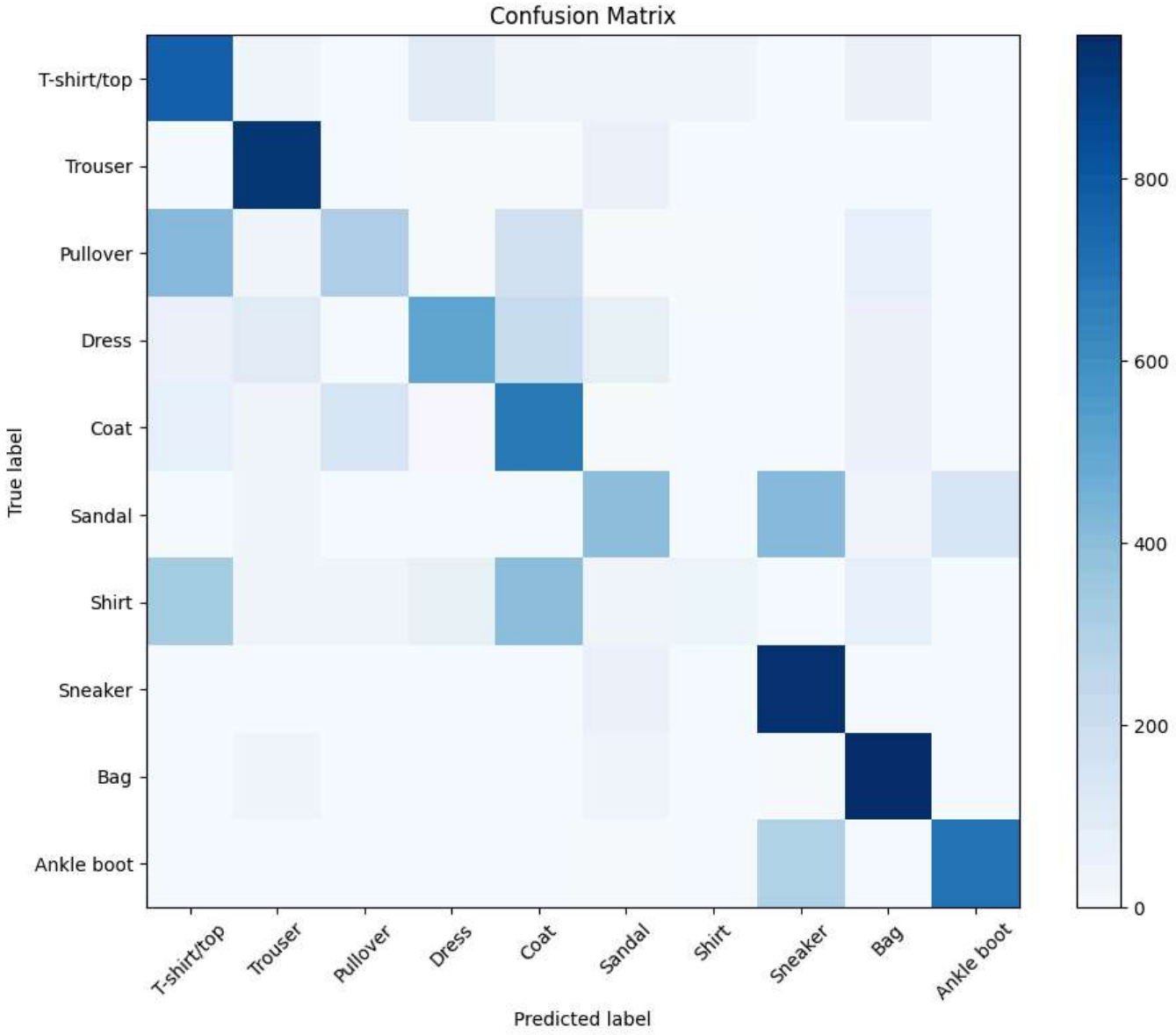
- ☐ Build text embeddings for the class names using the `get_text_embeddings` function (this only needs to be done once).
- ☐ For each batch of images:
  - ☐ Get image embeddings using the `get_image_embeddings` function.
  - ☐ Compute cosine similarity between the image and text embeddings using the `get_cosine_similarity` function.
  - ☐ Save the predictions so that we can build a confusion matrix later.
- ☐ Report the accuracy of the predictions and the confusion matrix using the `accuracy_score` and `confusion_matrix` functions from `sklearn.metrics`.

```
from sklearn.metrics import accuracy_score, confusion_matrix

y_true, y_pred = [], []

for pixel_values, labels in test_loader:
    # Get image embeddings
    image_feats = get_image_embeddings(pixel_values)

    # Get text embeddings
    text_feats = get_text_embeddings(CLASS_NAMES)

    # Compute cosine similarity
    similarity = get_cosine_similarity(image_feats, text_feats)

    # Get predictions
    preds = get_predictions(similarity)

    # save the predictions so that we can build the confusion matrix later
    y_true.extend(labels.cpu().numpy())
    y_pred.extend(preds)

# Report the accuracy of the predictions
accuracy = accuracy_score(y_true, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")

# Report the confusion matrix
def plot_confusion_matrix(y_true, y_pred, class_names):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(10, 8))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title('Confusion Matrix')
    plt.colorbar()
    tick_marks = np.arange(len(class_names))
    plt.xticks(tick_marks, class_names, rotation=45)
    plt.yticks(tick_marks, class_names)
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
```

```
    plt.tight_layout()
    plt.show()
```

```
Accuracy: 62.40%
```

```
plot_confusion_matrix(y_true, y_pred, CLASS_NAMES)
```



Reflection: Consider the results. How does the performance of this zero-shot baseline compare to the CNN you trained in Assignment 1? What are the strengths and weaknesses of this approach?

Accuracy looks to be low : 62.40% While it requires no task-specific training and is quick to deploy, it lacks the accuracy and task optimization that the CNN achieves through supervised learning.

Improving Zero-Shot Classification with Prompt Engineering

In the previous section, we directly used the class names as text prompts for zero-shot classification. However, we can often improve performance by crafting more descriptive prompts that better capture the visual characteristics of each class. For example, instead of just "T-shirt/top", we could use "a photo of a T-shirt" or "a photo of a top". This additional context can help the model make more accurate predictions.

In this section, we will experiment with more detailed prompts for each class to see if we can improve the zero-shot classification performance. You should:

- ☐ Create a list of improved prompts for each class. For example, instead of just "T-shirt/top", you could use "a photo of a T-shirt" or "a photo of a top".
- ☐ Use the `get_text_embeddings` function to create text embeddings for the improved prompts.
- ☐ Run the zero-shot classification baseline again using the improved prompts and report the accuracy and confusion matrix.

Note: Take advantage of the confusion matrix above. If two classes are often confused, consider how you might improve the prompts to help the model distinguish them better.

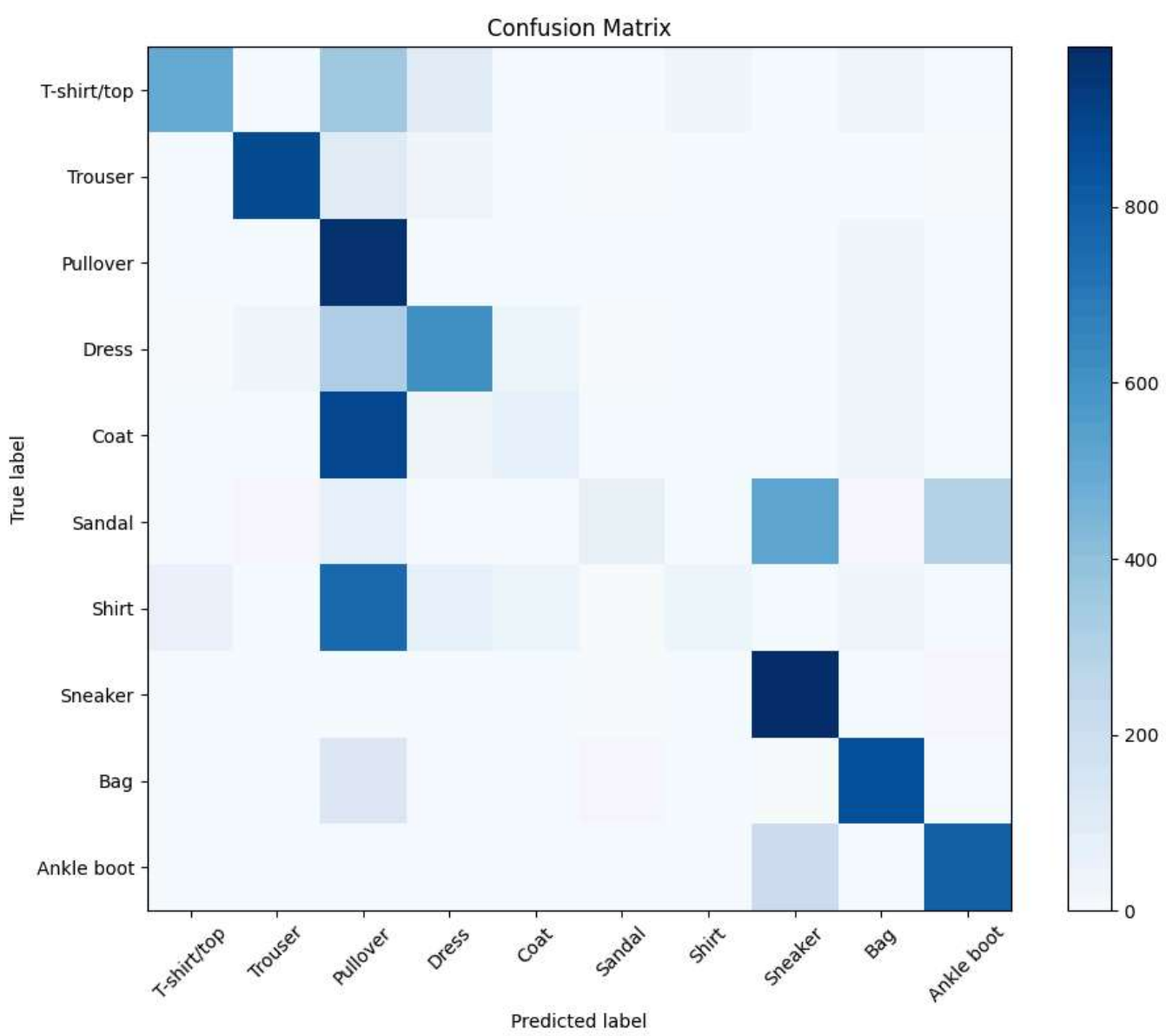
The aim for this section is for you to improve the performance of the model. However, if you find that the performance does not improve significantly, you can still reflect on the process and consider how you might further refine the prompts with more effort.

```
improved_prompts = [
    "a photo of a T-shirt",
    "a photo of trousers",
    "a photo of a pullover",
    "a photo of a dress",
    "a photo of a coat",
    "a photo of sandals",
    "a photo of a shirt",
    "a photo of sneakers",
    "a photo of a bag",
    "a photo of ankle boots"
]
```

```
#Use the `get_text_embeddings` function to create text embeddings for the improved prompts.
text_feats_improved = get_text_embeddings(improved_prompts)

#Run the zero-shot classification baseline again using the improved prompts and report the a
y_true_improved, y_pred_improved = [], []
for pixel_values, labels in test_loader:
    # Get image embeddings
    image_feats = get_image_embeddings(pixel_values)

    # Compute cosine similarity with improved text features
    similarity_improved = get_cosine_similarity(image_feats, text_feats_improved)

    # Get predictions
    preds_improved = get_predictions(similarity_improved)

    # save the predictions so that we can build the confusion matrix later
    y_true_improved.extend(labels.cpu().numpy())
    y_pred_improved.extend(preds_improved)

# Report the accuracy of the predictions with improved prompts
accuracy_improved = accuracy_score(y_true_improved, y_pred_improved)
print(f"Accuracy with improved prompts: {accuracy_improved * 100:.2f}%")
```

```
Accuracy with improved prompts: 57.76%
```

```
# Report the confusion matrix with improved prompts
plot_confusion_matrix(y_true_improved, y_pred_improved, CLASS_NAMES)
```



Reflection: How did your detailed prompts affect the zero-shot classification performance? Did you see a significant improvement compared to the baseline? What insights did you gain about the model's understanding of the classes? Do you think that with more effort you could further improve the performance? If so, how?

In my case, the accuracy got reduced a bit.

Visualizing Image Embeddings with UMAP

To better understand how the model perceives the different classes, we can visualize the image embeddings using UMAP (Uniform Manifold Approximation and Projection). UMAP is a dimensionality reduction technique that helps us see how similar or dissimilar the embeddings are in a lower-dimensional space.

By visualizing the embeddings, we can gain insights into how well the model can distinguish certain images, even without considering the text prompts. This can help us identify clusters of similar images and see if there are any overlaps between classes.

You should:

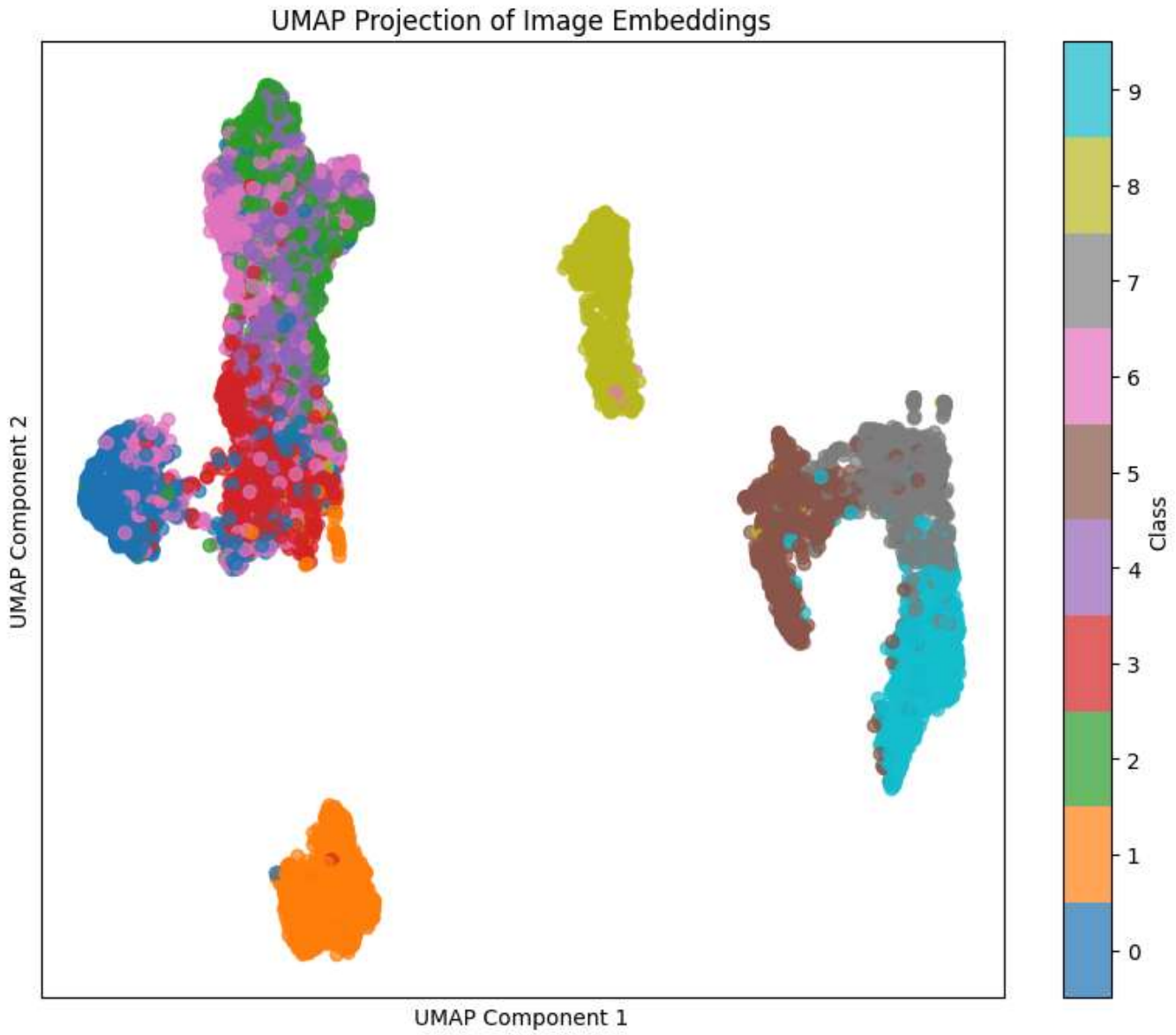
- ☐ Use the `get_image_embeddings` function to get the image embeddings for the entire test set.
- ☐ Use UMAP to reduce the dimensionality of the image embeddings to 2D.
- ☐ Plot the 2D embeddings, coloring each point by its true class label.

You may need to install the `umap-learn` library if you haven't already. You can do this by running `pip install umap-learn`.

```
# Uncomment the following line to install UMAP if you haven't already
# !pip install umap-learn


from umap import UMAP

# ------------------------------------------------------------
# 1. Collect image embeddings
# ------------------------------------------------------------
all_img_emb = []
all_labels  = []

for pixel_values, labels in test_loader:
    # Get image embeddings
    image_feats = get_image_embeddings(pixel_values)

    # Collect embeddings and labels
    all_img_emb.append(image_feats.cpu().numpy())
    all_labels.extend(labels.cpu().numpy())


# ------------------------------------------------------------
# 2. Fit UMAP
```

```
# ------------------------------------------------------------
umap_model = UMAP(n_neighbors=15, min_dist=0.1, n_components=2, random_state=42)
all_img_emb = np.concatenate(all_img_emb, axis=0)  # Concatenate all image embeddings
umap_embeddings = umap_model.fit_transform(all_img_emb)



# ------------------------------------------------------------
# 3. Plot coloured by ground-truth label
# ------------------------------------------------------------
def plot_umap(embeddings, labels, class_names):
    plt.figure(figsize=(10, 8))
    scatter = plt.scatter(embeddings[:, 0], embeddings[:, 1], c=labels, cmap='tab10', alpha=
    plt.colorbar(scatter, ticks=range(len(class_names)), label='Class')
    plt.clim(-0.5, len(class_names) - 0.5)
    plt.title('UMAP Projection of Image Embeddings')
    plt.xlabel('UMAP Component 1')
    plt.ylabel('UMAP Component 2')
    plt.xticks([])
    plt.yticks([])
    plt.show()

plot_umap(umap_embeddings, all_labels, CLASS_NAMES)
```

```
/usr/local/lib/python3.11/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1
  warn(
```



The UMAP embeddings allow us to see how separable or non-separable different classes are with our specific model. If two specific images are very similar, then they will be placed near each other on this graph.

Reflection: Do you notice any challenges in distinguishing images based on this figure? Are there any types of clothing in the dataset which the model has no trouble distinguishing from the others?

Mini-Experiment

In this section, you will conduct a mini-experiment of your choice to further explore the capabilities of zero-shot classification with transformers. This can be anything you'd like, but here are some ideas to get you started.

A. Alternative Model

So far we have been utilizing OpenAI's CLIP model for zero-shot classification. However, there are many other vision–language models available in the `transformers` library that you can experiment with. For example, there are larger CLIP models such as clip-vit-large-patch14, and open-source versions such as laion/CLIP-ViT-B-32-laion2B-s34B-b79K. You can also search huggingface here to find other models that might be suitable for zero-shot classification.

You can try using a different model to see if it improves the zero-shot classification performance. You should:

- ☐ Load a different model and processor from the `transformers` library.
- ☐ Run the zero-shot classification baseline with the new model and report the accuracy and confusion matrix.
- ☐ Reflect on the performance of the new model compared to the original CLIP model
  - How does the new model perform compared to the original CLIP model?
  - Do you notice any differences in the types of errors made by the new model?

B. Multiple-Description Classification

Another interesting experiment is to explore multiple-description classification. *This involves providing multiple text prompts for each class, allowing the model to choose the most relevant one. For example, instead of just "T-shirt/top", you could provide "a photo of a T-shirt", "a photo of a top", and "a photo of a shirt". This can help the model better understand the class and increases the likelihood of a correct prediction. You should:

- ☐ Create a list of multiple prompts for each class.
- ☐ Use the `get_text_embeddings` function to create text embeddings for the multiple prompts.
- ☐ Run the zero-shot classification baseline again using the multiple prompts and report the accuracy and confusion matrix.
- ☐ Consider the model to be correct if it guesses *any* of the prompts belonging to the correct class.

C. Top-K Classification

In some classification tasks, it can be useful to consider if the right answer is among the top K (e.g. top 3) predictions. This can be particularly useful in cases where the model is uncertain or when

there are multiple similar classes. You should:

- ☐ Modify the `get_predictions` function to return the top K predictions for each image.
- ☐ Modify the accuracy calculation to consider the model correct if the true class is among the top K predictions.
- ☐ Report the accuracy and confusion matrix for the top K predictions. Report at least two different values of K (e.g. K=2 and K=4).

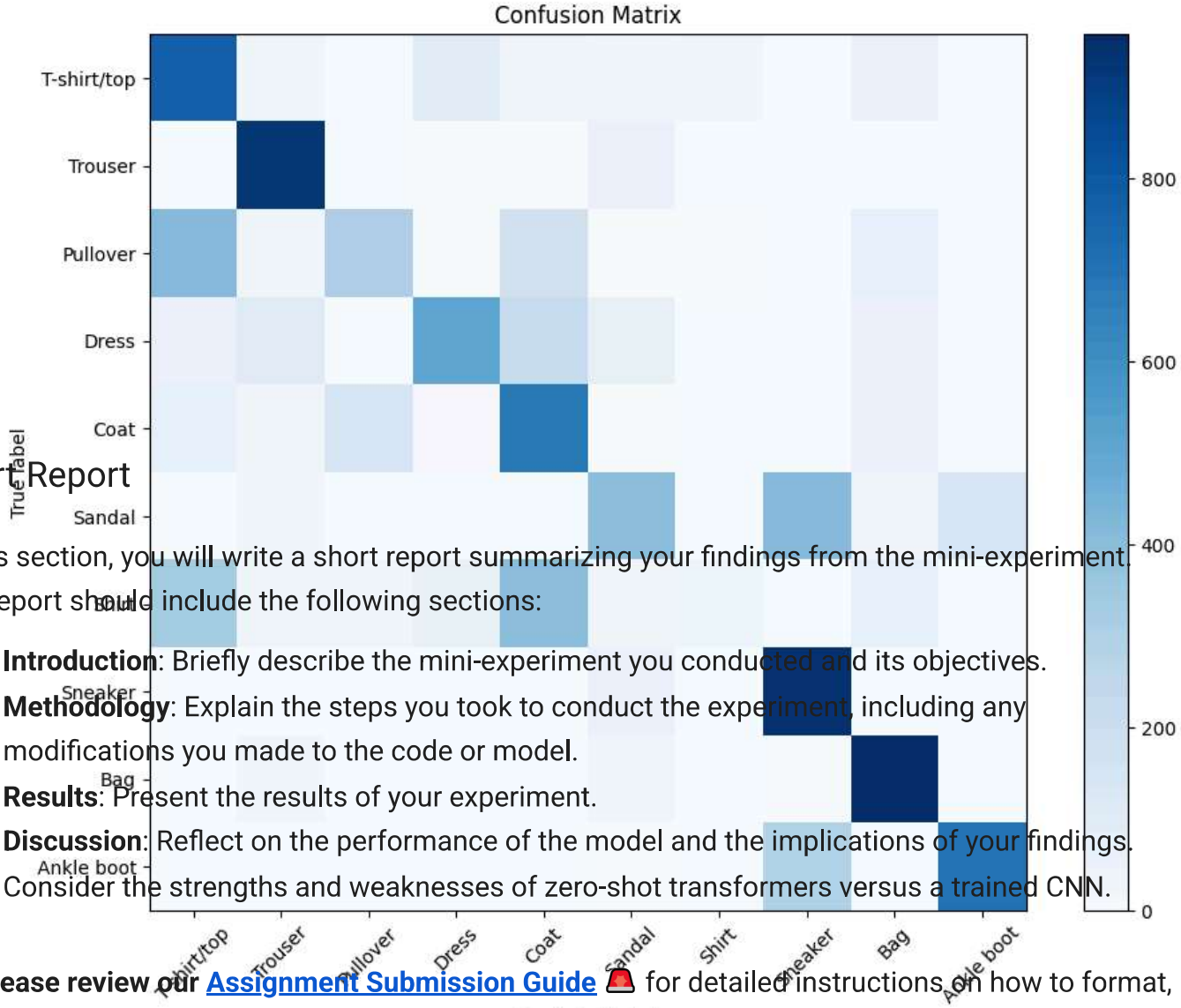
D. Other Ideas

You are welcome to come up with your own mini-experiment! Explain your idea in the report and implement it. Did it work as you expected? What did you learn from it?

```
# Top K classification
# Modify the `get_predictions` function to return the top K predictions for each image.
def get_top_k_predictions(similarity: np.ndarray, k: int = 3) -> np.ndarray:
    """
    Get the top K predictions based on cosine similarity scores.
    Args:
        similarity (numpy.ndarray): Cosine similarity matrix of shape (N, M).
        k (int): Number of top predictions to return.
    Returns:
        numpy.ndarray: Indices of the top K predicted classes for each image, shape (N, K).
    """
    return np.argsort(similarity, axis=1)[:, -k:][:, ::-1]  # Descending order

# Modify the accuracy calculation to consider the model correct if the true class is among t
def top_k_accuracy(y_true: list[int], y_pred_topk: np.ndarray, k: int) -> float:
    """
    Calculate top-K accuracy.
    Args:
        y_true (list[int]): True class labels.
        y_pred_topk (np.ndarray): Predicted top K class indices, shape (N, K).
        k (int): K value used in prediction.
    Returns:
        float: Top-K accuracy score.
    """
    correct = sum(y in y_pred_topk[i] for i, y in enumerate(y_true))
    return correct / len(y_true)

# Report the accuracy and confusion matrix for the top K predictions. Report at least two di
from sklearn.metrics import confusion_matrix, accuracy_score
import matplotlib.pyplot as plt

def evaluate_top_k(k_values=[1, 2, 4]):
    y_true = []
    top_k_preds = {k: [] for k in k_values}

    for pixel_values, labels in test_loader:
```

```
            image_feats = get_image_embeddings(pixel_values)
            text_feats = get_text_embeddings(CLASS_NAMES)
            similarity = get_cosine_similarity(image_feats, text_feats)

            for k in k_values:
                preds = get_top_k_predictions(similarity, k=k)
                top_k_preds[k].extend(preds)
            y_true.extend(labels.cpu().numpy())

        # Accuracy and Confusion Matrix
        for k in k_values:
            acc = top_k_accuracy(y_true, np.array(top_k_preds[k]), k)
            print(f"Top-{k} Accuracy: {acc * 100:.2f}%")

            if k == 1:
                # Only show confusion matrix for Top-1
                plot_confusion_matrix(y_true, np.array(top_k_preds[k]).squeeze(), CLASS_NAMES)

    evaluate_top_k(k_values=[1, 2, 4])
```

```
Top-1 Accuracy: 62.40%
```



Short Report

In this section, you will write a short report summarizing your findings from the mini-experiment. The report should include the following sections:

- **Introduction**: Briefly describe the mini-experiment you conducted and its objectives.
- **Methodology**: Explain the steps you took to conduct the experiment, including any modifications you made to the code or model.
- **Results**: Present the results of your experiment.
- **Discussion**: Reflect on the performance of the model and the implications of your findings. Consider the strengths and weaknesses of zero-shot transformers versus a trained CNN.

🚨 **Please review our Assignment Submission Guide** 🚨 for detailed instructions on how to format,